PID Control — Concept and Application in This Lab

What is PID Control?

PID control stands for **Proportional–Integral–Derivative control**.
It's one of the most widely used control algorithms in engineering because it's simple, robust, and works well for a large range of systems.

A PID controller continuously measures the **error** between a desired target (**setpoint**) and the actual measured value (**process variable**) and then adjusts the **control input** to minimize this error over time.

$$\text{error}(t) = \text{Setpoint}(t) - \text{ProcessVariable}(t)$$

The controller combines three actions — **P**, **I**, and **D** — each with a different role. By tuning these three terms, we can make the system respond **quickly**, **accurately**, and **stably**.

Why We Use PID Control in This Project

In this lab, our **goal** is to keep the LED's brightness (measured by the LDR) equal to a target level chosen by the user (the setpoint).
However, the relationship between PWM duty cycle and measured brightness is **not perfectly linear**, and there are **delays** and **disturbances** (e.g., ambient light, sensor noise).

If we simply used **on/off control**, the LED would flicker and overshoot badly.
If we used **P-only control**, there would still be some steady-state error.

PID control solves this by:

- Reacting **proportionally** to current error,
- **Accumulating** past error to eliminate offsets,
- **Predicting** future trends to smooth and stabilize the response.

This is why PID is a perfect fit for our closed-loop LED–LDR system.

Responsibilities of Each Term

Term	Role	Effect	Typical Problems if Misused
P - Proportional	Reacts to the **current error**	Larger error → stronger correction. Speeds up response.	If too high → overshoot & oscillation. If too low → slow response & steady error.
I -Integral	Reacts to the **accumulated error over time**	Eliminates **steady-state error** (e.g., small offset that P alone can't fix).	If too high → slow oscillations, overshoot, "windup."
D - Derivative	Reacts to the **rate of change of the measurement**	Predicts where the system is going; helps **dampen overshoot** and stabilize response.	If too high → amplifies noise, makes output jittery.

Mathematically (continuous form):

$$u(t) = K_p\, e(t) \;+\; K_i \int_0^t e(\tau)\, d\tau \;+\; K_d \frac{de(t)}{dt}$$

In our Arduino code, we use a **discrete version** of this equation that runs every sampling period Ts.

Important Design Points for This PID Controller

When designing and coding your controller, pay close attention to the following:

1. **Consistent Units**
    - Make sure setpoint, measurement (LDR), and control output (PWM) use the same scale (e.g. 0–255).
    - Inconsistent scaling leads to unstable or sluggish control.
2. **Sampling Time (Ts)**
    - Choose Ts to be fast enough to react to system changes but not so fast that noise dominates.
    - For LED/LDR, Ts ≈ 50 ms works well.

3. **Filtering Sensor Noise**
    - Use an exponential moving average (EMA) to smooth LDR readings before using them in PID calculations.
    - This improves stability and makes D-term useful.
4. **Integral Anti-Windup**
    - If the output saturates at 0 or 255 and the error persists, the integral term can "wind up," causing large overshoots when the error changes sign.
    - Always implement a condition to pause integral accumulation during saturation.
5. **Derivative on Measurement**
    - We compute derivative on the **measured value** instead of the error to reduce noise spikes when the setpoint changes suddenly.
6. **System Polarity**
    - Make sure increasing PWM actually increases measured brightness.
    - If it doesn't, invert your LDR mapping or error calculation to get the correct control direction.
7. **Tuning Order**
    - Start with **P only** → increase until you get a fast but slightly overshooting response.
    - Add **I** slowly to remove steady-state error.
    - Add a small **D** to reduce overshoot and jitter.
    - Adjust gradually and observe the effect of each term.
8. **Step & Disturbance Testing**
    - Step the setpoint up and down and observe the system's response.
    - Introduce a disturbance (e.g., block the LDR with your hand) to test recovery.

PID control is a powerful way to make a system automatically reach and maintain a desired target value.

- **P** gets you moving toward the target,

- **I** gets you all the way there and removes steady errors,
- **D** keeps the response smooth and stable.

By carefully tuning and implementing these three components, your LED–LDR system can behave like a well-controlled feedback system — fast, stable, and accurate.